3. La vista

Todo lo de `src/view/`. Su único trabajo es **enseñar una imagen en una ventana**. No sabe nada de fuego ni de temperaturas: solo recibe imágenes.

Son dos clases con trabajos distintos:

Clase	Qué es	Su trabajo
`View`	La ventana (`JFrame`) y el hilo	Pide imágenes y las manda a dibujar
`Viewer`	El lienzo (`Canvas`)	Dibuja la imagen que le den

Imagínalo como un marco con un cuadro dentro: `View` es el marco (y el que cambia el cuadro), `Viewer` es el lienzo donde se pinta.

Viewer: el lienzo

`Viewer.java`

```
public class Viewer extends Canvas
```

`extends Canvas` significa que `Viewer` **es** un lienzo de Java, y hereda todo lo que sabe hacer uno.

`setImage()`

`Viewer.java:10`

```
this.image = image;
repaint();
```

Guarda la imagen y llama a `repaint()`. `repaint()` **no dibuja** directamente: avisa a Java de que hay que redibujar, y Java llama a `update()` cuando puede.

`paint()`: dibujar

`Viewer.java:18`

```
g.drawImage(image, 0, 0, getWidth(), getHeight(), null);
```

Dibuja la imagen desde la esquina (0, 0), **estirada** hasta ocupar todo el lienzo (`getWidth()` × `getHeight()`). Por eso, si agrandas la ventana, el fuego se agranda con ella.

`update()` : evitar el parpadeo

`Viewer.java:24`

```
public void update(Graphics g){ paint(g); }
```

Por defecto, `Canvas` **borra todo el lienzo** antes de dibujar. En una animación eso se nota como un parpadeo. Al sobrescribir `update()` para que solo llame a `paint()` , no se borra nada y la imagen nueva se dibuja directamente encima.

View: la ventana

`View.java`

```
public class View implements IView, Runnable
```

Implementa dos cosas: - `IView` : el contrato con el Presenter (`setPresenter` y `start`). - `Runnable` : significa que tiene un método `run()` que puede ejecutar un hilo.

Los atributos

`View.java:11-16`

Atributo	Para qué
`presenter`	Pedirle las imágenes. Empieza vacío y se asigna después
`frame`	La ventana
`viewer`	El lienzo que va dentro de la ventana
`FRAME_MS = 16`	Espera entre refrescos: 1000 / 16 ≈ **60 veces por segundo**

El constructor

`View.java:18`

```
viewer = new Viewer();
frame = new JFrame("Fire Animation");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.add(viewer);
```

Crea el lienzo y la ventana, y mete el lienzo dentro. `EXIT_ON_CLOSE` hace que al cerrar la ventana termine el programa. **Todavía no se muestra nada**: eso pasa en `start()`.

`setPresenter()`

`View.java:27`

La vista no recibe el Presenter en el constructor, sino después. Es porque el Presenter se crea **después** de la vista (necesita la vista para existir). Es el `Presenter.start()` quien llama a `view.setPresenter(this)`.

`start()`: abrir la ventana

`View.java:32`

Paso a paso:

```
BufferedImage first = presenter.getImage();
```

1. Pide una primera imagen. Se necesita para saber el tamaño (320 × 200).

```
viewer.setPreferredSize(new Dimension(first.getWidth(), first.getHeight()));
viewer.setImage(first);
```

1. Le dice al lienzo que quiere ser de ese tamaño y le da la primera imagen.

```
frame.pack();
frame.setVisible(true);
```

1. `pack()` ajusta la ventana al tamaño de lo que lleva dentro. `setVisible(true)` por fin la muestra.

```
Thread thread = new Thread(this, "fire-view");
thread.setDaemon(true);
thread.start();
```

1. Arranca el hilo que refresca la imagen. Igual que en el modelo, el `this` hace que el hilo ejecute el `run()` de `View`.

`run()`: el bucle de refresco

`View.run()`

```
while (true) {
    viewer.setImage(presenter.getImage());
    Thread.sleep(FRAME_MS);   // 16 ms
}
```

Repite para siempre: pedir imagen nueva, mandarla a dibujar y esperar 16 ms. Esto es lo que hace que veas el fuego moverse.

Las dos velocidades

	Cada cuánto	Veces por segundo
El fuego avanza (`Model`)	33 ms	~30
La ventana se refresca (`View`)	16 ms	~60

Son independientes. La ventana pide imágenes más rápido de lo que el fuego cambia, así que a veces repinta la misma imagen. No es un problema; solo es un refresco más suave.